

Sorting

Sorting is any process of arranging items in a sequence ordered by some criterion (increasing or decreasing). Sorting can be done on names, numbers and records. Sorting reduces the For example, it is relatively easy to look up the phone number of a friend from a telephone dictionary because the names in the phone book have been sorted into alphabetical order.

Different algorithms are available to perform sorting operation. Some of the notable ones are:

- Bubble sort : Exchange two adjacent elements if they are out of order. Repeat until array is sorted.
- Insertion sort : Scan successive elements for an out-of-order item, then insert the item in the proper place.
- Selection sort : Find the smallest element in the array, and put it in the proper place. Swap it with the value in the first position. Repeat until array is sorted.
- Quick sort : Partition the array into two segments. In the first segment, all elements are less than or equal to the pivot value. In the second segment, all elements are greater than or equal to the pivot value. Finally, sort the two segments recursively.
- Merge sort : Divide the list of elements in two parts, sort the two parts individually and then merge it.

Bubble Sort

It's a sorting algorithm, in which each pair of adjacent items are compared and swapped if they are in wrong order. The comparison is repeated until no swaps are needed, indicating that the list is sorted. The smaller elements 'bubble' to the top of the list, hence, the name Bubble Sort. In this like selection sort, after every pass the largest element moves to the highest index position of the list.

Algorithm:

It starts with the first element of the list. The comparison of the adjacent pair of elements and swapping is done until the list is sorted as follows

1. Compare two adjacent elements and check if they are in correct order (that is second one has to be greater than the first).
2. Swap them if they are not in correct order.

Let iter denotes the number of iterations, then for iter ranging from 1 to n-1 (where n is total number of elements in the list) check

1. If value of the second item at position iter is lesser than the value of the first item i.e. at position iter-1, then swap them.
2. Else, move to the next element at position iter+1.

The effective size of the list is hence reduced by one after every pass and the largest element is moved to its final position in the sorted array. Repeat the two steps until the list is sorted and no swap is required i.e. the effective size is reduced to 1.

Properties:

1. Best Case performance – When the list is already sorted, we require only one pass to check, hence $O(n)$.
2. Worst Case performance – When the list is in the reverse order, n number of comparisons and swap are required to be done n number of times, hence $O(n^2)$.
3. Average Case performance – $O(n^2)$

To trace the algorithm with an example,

Refer to Attachment: Arrays_Bubble_Sort.pdf

Program:

```
#include<stdio.h>
```

```
/* Logic : Do the following thing until the list is sorted
```

```
    (i) Compare two adjacent elements and check if they are in correct order(that is  
    second one has
```

```
        to be greater than the first).
```

```
    (ii) Swap them if they are not in correct order.
```

```
*/
```

```
int main()
```

```
{
```

```
    int number_of_elements;
```

```
    printf("Enter the no of elements\n");
```

```
    scanf("%d",&number_of_elements);
```

```
    int array[number_of_elements];
```

```
    int iter;
```

```

printf("Enter the elements\n");

for(iter = 0;iter < number_of_elements;iter++)

{

    scanf("%d",&array[iter]);

}

/* Calling this functions sorts the array */

int temp, swapped;

do

{

    swapped = 0; /* If no element is swapped array is sorted */

    /* In the following loop compare every pair of adjacent elements and check

    if they are in correct order */

    for(iter = 1; iter < number_of_elements; iter++)

    {

        if(array[iter-1] > array[iter])

        {

            temp = array[iter-1];

            array[iter-1] = array[iter];

            array[iter] = temp;

            swapped = 1;

        }

    }

}

}while(swapped);

printf("Elements after sorting is as follows: \n");

for(iter = 0;iter < number_of_elements;iter++)

{

```

```
        printf("%d ",array[iter]);  
    }  
    printf("\n");  
    return 0;  
}
```

Selection Sort

In this technique, the first element is selected and compared with all other elements. If any other element is less than the first element swapping should take place. By the end of this comparison, the least element must top position in the array. This is known as pass1. In pass II, the second element is selected and compared with all other elements. Swapping takes place if any other element is less than selected element. This process continues until array is sorted.

Property:

1. Best case performance – When the list is already sorted $O(n^2)$.
2. Worst case performance - When the list is sorted in reverse order $O(n^2)$.
3. Average case performance – $O(n^2)$.
4. $O(n^2)$ time complexity makes it difficult for long lists.

Algorithm:

1. Find the minimum element in the list.
2. Swap it with the element in the first position of the list.
3. Repeat the steps above for all remaining elements of the list starting from the second position.
4. The iteration stops when no more swap is further required.

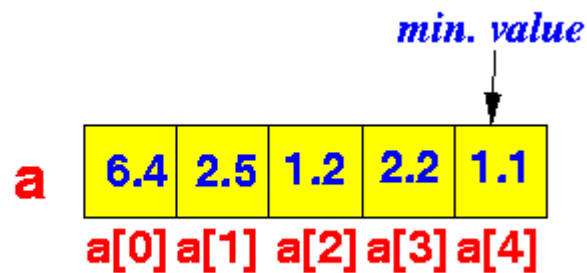
The *Selection Sort* algorithm - development step

- Pseudo code of the *Selection Sort* algorithm:

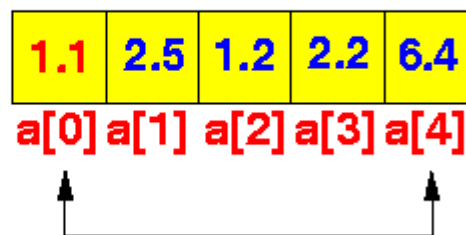
```
Let: a = array containing the values
Let: n = # of elements

1. Find the array element with the min. value among a[0],
a[1], ..., a[n-1];
2. Swap this element with a[0]
```

Illustration:



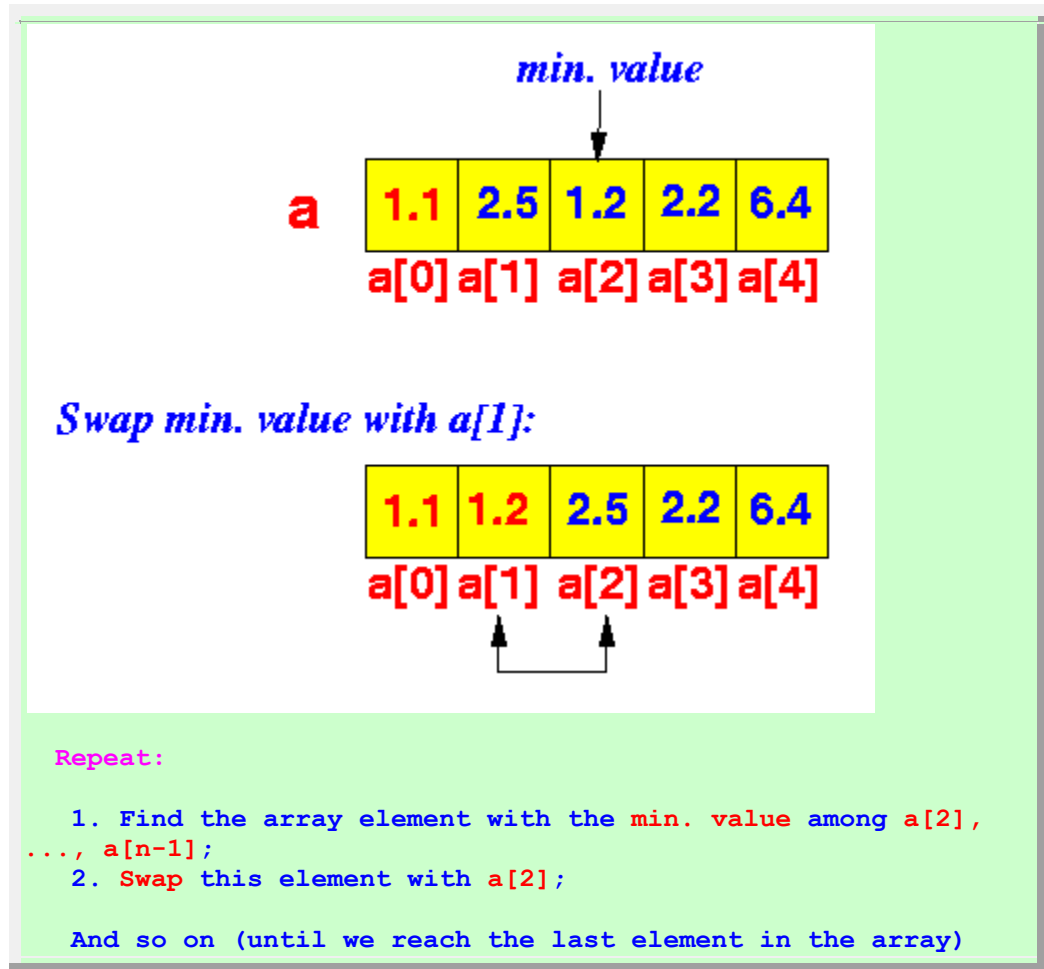
Swap min. value with a[0]:



Repeat:

```
1. Find the array element with the min. value among a[1],
..., a[n-1];
2. Swap this element with a[1];
```

Illustration:



Program:

```
#include<stdio.h>
```

```
int main(){
```

```
    int s,i,j,temp,a[20];
```

```
    printf("Enter the limit: ");
```

```
    scanf("%d",&s);
```

```
    printf("Enter %d elements: \n",s);
```

```
    for(i=0;i<s;i++)
```

```
        scanf("%d",&a[i]);
```

```
    for(i=0;i<s;i++){
```

```
    for(j=i+1;j<s;j++){  
        if(a[i]>a[j]){  
            temp=a[i];  
            a[i]=a[j];  
            a[j]=temp;  
        }  
    }  
}
```

```
printf("After sorting is: \n");  
for(i=0;i<s;i++)  
    printf(" %d\n",a[i]);  
return 0;  
}
```